FLUTTER COURSE · SESSION 5 OF 15

State Management with Riverpod

With `setState`, data lives inside one screen, and every other widget that needs it has to receive it through constructors. Today you learn Riverpod, which keeps shared data in **providers** that any widget on any screen can read and update directly.

BY THE END OF TODAY YOU CAN

Share state between screens with Riverpod, pick the right kind of provider, and keep your app's rules in a Notifier instead of inside widgets.

Setup

```
flutter pub add flutter_riverpod
```

This adds `flutter_riverpod: ^3.3.2` to `pubspec.yaml`. In VS Code, today's apps are in the **Session 5** group of the Run and Debug dropdown. From the terminal:

```
flutter run -t lib/day5_demos/demo1_setstate_problem.dart
flutter run -t lib/pizza_order/main.dart
```

RIVERPOD 3 NOTE

`StateProvider` moved to a separate import in Riverpod 3. If you see “StateProvider isn't defined”, add `import 'package:flutter_riverpod/legacy.dart';`. Many older tutorials and AI answers miss this.

Riverpod words, in plain English

Think of a classroom. With `setState`, the data is a note in one person's pocket: to reach anyone else, it has to be passed hand to hand. With Riverpod, the data is pinned on a **notice board on the wall**. Anyone can look at the board, and only the board's caretaker is allowed to change it.

WORD	IN THE STORY	WHAT IT REALLY IS
`ProviderScope`	The school building that holds all the notice boards	Wraps your whole app, once, in `main()`. It stores the value of every provider. Without it the app crashes.
provider `cartProvider`	One notice board, with a name	A piece of shared data, declared at the top of a file, outside every class, so any widget can find it by name.
`ConsumerWidget`	A student who can see the boards	A `StatelessWidget` that can use providers. Its `build` gets an extra `WidgetRef ref`.
`ConsumerStatefulWidget`	The same, for a student who also keeps their own notes	A `StatefulWidget` that can use providers. Use it when you also need `initState` / `dispose`, e.g. for a `TextEditingController`. Here `ref` is available everywhere in the State class.
`ref`	The student's eyes and hands for the boards	The object a widget uses to reach providers. No `ref`, no providers.
`ref.watch(p)`	Keep looking at the board, and update your notebook whenever it changes	Returns the value **and rebuilds this widget whenever it changes**. Use it inside `build()`. This is what replaces `setState`.
`ref.read(p)`	Walk up to the board once	Returns the value right now, without listening. Use it inside callbacks such as `onPressed`.
`p.notifier`	The board's caretaker	The object that can **change** the value. Without `.notifier` you get the value itself; with it you get the thing that changes it.
`state`	The sheet currently pinned on the board	The provider's current value. Assigning `state = …` pins up a new sheet, and every watcher rebuilds.

`Notifier`	A caretaker with a fixed list of jobs	A class you write with methods such as `add` and `remove`. They are the only way to change that provider.
`build()` in a Notifier	What's on the board on the first day	Returns the starting value, e.g. an empty list. Runs once, like `initState`.

Reading one line of Riverpod

```
ref.read(cartProvider.notifier).add(name)
```

PART	SAY IT AS
`ref`	“Using my access to the boards…”
`.read(…)`	“…walk up once (I'm in a button press, I don't need to keep watching)…”
`cartProvider`	“…to the cart board…”
`.notifier`	“…and talk to its caretaker…”
`.add(name)`	“…and ask it to add this product.”

THE TWO RULES

1. `watch` to show, `read` to change. In `build()`, always `ref.watch`. In `onPressed` and other callbacks, use `ref.read`. A widget rebuilds only if it **watches** something that changed.

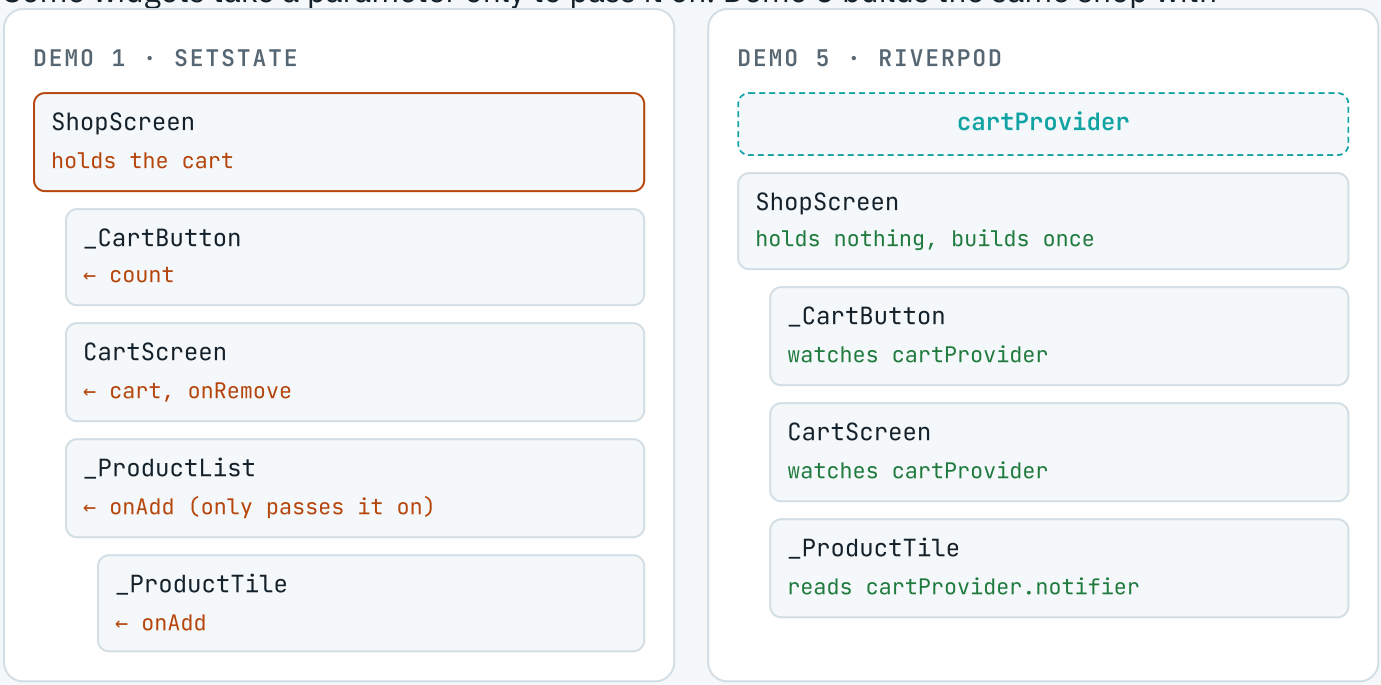
2. Always give `state` a new value. Write `state = [...state, item]`, never `state.add(item)`. Riverpod only notices a new list or a new object.

The concepts, in the order we cover them

`demo1` Why setState breaks down

In demo 1 the cart lives in `ShopScreen`'s State. Every widget that needs the cart, or needs to change it, must receive it through a constructor. This is called **prop drilling**.

Some widgets take a parameter only to pass it on. Demo 5 builds the same shop with



Problems with the setState version:

- Data and callbacks pass through widgets that don't use them.
- When the cart screen changes the cart, both screens need extra `setState` calls to redraw.
- The whole `ShopScreen` rebuilds for every change, even parts that didn't change.

A rebuild takes about a millisecond, so you can't normally see it. In demos 1 and 5, every widget shows a grey **"built 1×"** tag that counts its rebuilds and flashes yellow each time. In demo 1, tap **+** on one product and **all eight tags** go up.

`demo2` Setup: ProviderScope, ConsumerWidget and Provider

Nothing changes in demo 2. It shows **where** a shared value lives and **how** any widget reads it. Without Riverpod, you'd pass the app name down by hand:

```
// Without Riverpod: passed through every constructor
HomeScreen(appName: 'Campus Store')
  └─ _Footer(appName: appName)      // receives it again, and so on
```

With Riverpod, nothing is passed. `_Footer()` reads the name itself. Using Riverpod takes three steps.

```dart
// 1. Declare a provider at the top level of a file.
final appNameProvider = Provider<String>((ref) => 'Campus Store');

// 2. Wrap the app in ProviderScope. It stores every provider's value.
void main() => runApp(const ProviderScope(child: MyApp()));

// 3. Read it in a ConsumerWidget through `ref`.
class HomeScreen extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final appName = ref.watch(appNameProvider);
    return Text(appName);
  }
}
```

- `Provider` holds a value that doesn't change, or one calculated from other providers.
- `ConsumerWidget` is a `StatelessWidget` with a `ref`. `ConsumerStatefulWidget` is the same for a `StatefulWidget`.
- If you forget `ProviderScope`, the app throws `Bad state: No ProviderScope found`.

`demo3` StateProvider, and ref.watch vs ref.read

`StateProvider` holds one simple value that can change, like a counter or an on/off switch.

```dart
import 'package:flutter_riverpod/legacy.dart';

final counterProvider = StateProvider<int>((ref) => 0);

// in build:
final count = ref.watch(counterProvider);

// in a button:
onPressed: () => ref.read(counterProvider.notifier).state++,
```

`ref.watch`

“Rebuild me whenever this value changes.”

`ref.read`

“Give me the value or notifier right now, once.”

Use it **inside** `build()` for anything you display.

Use it **inside callbacks** like `onPressed` to change state.

In the demo, replace `ref.watch` with `ref.read` on the commented line in `build`.

The number stops updating on screen.

`demo4` NotifierProvider for complex state

When state has several operations, such as add, toggle and remove, put them in a **Notifier** class. Widgets call its methods, and all the rules live in one place.

```
class TaskListNotifier extends Notifier<List<Task>> {
  @override
  List<Task> build() => [];              // the starting state

  void add(String title) {
    state = [...state, Task(title: title)];   // a NEW list
  }

  void remove(int id) {
    state = state.where((t) => t.id != id).toList();
  }
}

final taskListProvider =
    NotifierProvider<TaskListNotifier, List<Task>>(TaskListNotifier.new);

// in a widget:
final tasks = ref.watch(taskListProvider);               // the list
ref.read(taskListProvider.notifier).add('Revise Dart');    // call a method
```

State must be replaced, not changed

Riverpod only notices a change when you **assign a new value** to `state`. Changing the old list in place does nothing on screen.

```
state.add(task);                // ❌ same list, Riverpod sees no change
state = [...state, task];       // ✅ new list, widgets rebuild

task.isDone = true;             // ❌ fields should be final
task.copyWith(isDone: true)     // ✅ a new Task with one field changed
```

Derived providers

Don't store a value you can calculate. A `Provider` that watches another provider recalculates automatically:

```
final remainingCountProvider = Provider<int>((ref) {
  return ref.watch(taskListProvider).where((t) => !t.isDone).length;
});
```

`demo5` Sharing state across screens

Demo 5 is the shop from demo 1, rebuilt with a `cartProvider`. The two apps look and work the same; only the inside is different.

```
// The caretaker: the only way to change the cart
class CartNotifier extends Notifier<List<String>> {
  @override
  List<String> build() => const [];                 // starts empty
  void add(String item) => state = [...state, item];
  void remove(String item) => state = [...state]..remove(item);
}
final cartProvider = NotifierProvider<CartNotifier, List<String>>
(CartNotifier.new);

// Product tile: CHANGES the cart, never watches it, so it never rebuilds
onPressed: () => ref.read(cartProvider.notifier).add(name),

// Cart button: SHOWS the count, so it rebuilds when the cart changes
final count = ref.watch(cartProvider).length;

// Cart screen: shows the list AND removes items, with no callbacks passed in
final cart = ref.watch(cartProvider);
onPressed: () => ref.read(cartProvider.notifier).remove(item),
```

- `ShopScreen` is a plain `StatelessWidget`. It holds nothing and passes nothing.
- Remove an item on the cart screen and the badge on the shop screen updates by itself. There are no `setState` calls anywhere.

Do the same taps in both demos and compare the “built” tags:

DO THIS	DEMO 1 · SETSTATE	DEMO 5 · RIVERPOD
Add Pen	all 8 widgets → 2×	only the cart button → 2×
Add Backpack	all 8 widgets → 3×	only the cart button → 3×
Open the cart, delete Pen	cart screen → 2×	cart screen → 2×, cart button → 4×
Go back to the shop	all 8 widgets → 5×	title and tiles still 1×
`setState` calls written by hand	4	0

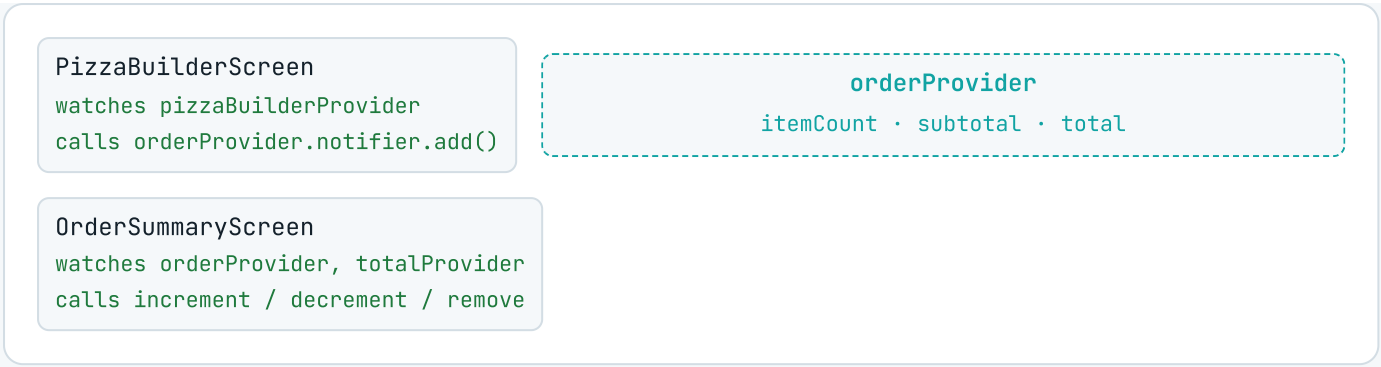
Only the widgets that `watch` the cart (the cart button and the cart screen) ever rebuild.

Which provider should I use?

PROVIDER	USE IT FOR	EXAMPLE
`Provider`	A value that never changes, or one calculated from other providers	App name, tax rate, menu items, cart total
`StateProvider`	One simple value changed from outside	Counter, dark mode on/off, home delivery switch
`NotifierProvider`	State with several operations and rules	Cart, task list, the pizza being built

The project: Pizza Order App

On the first screen you pick a size, any toppings and a quantity. The button shows the live price, for example “Add 2 to order · ₹918”. The second screen lists everything in the order: change quantities, remove pizzas, switch on home delivery, and see the item count and total update as you go. A badge on the first screen's AppBar always shows how many pizzas are in the order.




```
lib/pizza_order/
├─ main.dart · ProviderScope, named routes
├─ routes.dart · class AppRoutes
├─ models/pizza.dart · immutable classes with copyWith
├─ providers/
│  ├─ menu_providers.dart · sizes and toppings
│  ├─ pizza_builder_provider.dart · the pizza being built
│  └─ order_provider.dart · the order and its totals
├─ screens/ · pizza_builder_screen.dart, order_summary_screen.dart
└─ widgets/ · order_button.dart, quantity_stepper.dart
```

Where each concept appears

FILE	WHAT TO LOOK FOR
`menu_providers.dart`	Plain `Provider`s for read-only menu data
`pizza_builder_provider.dart`	A `Notifier` with `selectSize`, `toggleTopping`, `increment` and `decrement`. `reset()` calls `ref.invalidateSelf()` to start over.
`order_provider.dart`	A `Notifier` for the order, a `StateProvider` for home delivery, and derived providers for item count, subtotal, delivery charge and total
`pizza_builder_screen.dart`	`ref.watch` in `build`, `ref.read(...notifier)` in button callbacks

`order_summary_screen.dart`	Changing a quantity here updates the badge on the other screen, with no data passed between them
`order_button.dart`	A small `ConsumerWidget`, so only the badge rebuilds when the order changes

Testing the rules without any screens

Because the rules live in providers, `test/pizza_order_test.dart` checks them with a `ProviderContainer` and no widgets at all:

```
final container = ProviderContainer();
container.read(orderProvider.notifier).add(draft);
expect(container.read(itemCountProvider), 2);
expect(container.read(subtotalProvider), 598);
```

Run `flutter test` to see it pass.

Errors you will see today

> **“Bad state: No ProviderScope found”**
>
> Wrap your app in `ProviderScope` inside `main()`.

> **`StateProvider isn't defined`**
>
> Riverpod 3 moved it. Add `import 'package:flutter_riverpod/legacy.dart';`.

> **`The screen doesn't update after calling a notifier method`**
>
> You changed the state in place (`state.add(...)`). Assign a new value: `state = [...state, item]`.

> **`The screen never updates`**
>
> You used `ref.read` inside `build`. Use `ref.watch` there.

> **`ref.read(cartProvider).add(item)` runs but nothing changes**
>
> Without `.notifier` you get the list itself, and `add` changes it in place. Call the notifier's method: `ref.read(cartProvider.notifier).add(item)`.

> **“Undefined name 'ref'”**
>
> The widget still extends `StatelessWidget`. Change it to `ConsumerWidget` and add `WidgetRef ref` to `build`.

Practice task (in class)

Build a **Counter + Cart** app with Riverpod.

- [ ] Screen 1 lists products, each with **Add** and **Remove** buttons.
- [ ] Screen 2 shows the live item count and total price.
- [ ] Screen 1's AppBar shows the item count as a badge.
- [ ] The cart is a `NotifierProvider`. The count and total are derived `Provider`s.
- [ ] No cart data is passed through any constructor.

Homework: extend the Pizza Order App

- [ ] Add a “Veg only” switch (`StateProvider<bool>`) that hides Chicken from the toppings, using a derived `Provider`.
- [ ] Add a coupon: `PIZZA50` gives ₹50 off orders over ₹500. Put the rule in a provider, not in a widget.
- [ ] Make delivery free for orders over ₹999.
- [ ] Write a unit test with `ProviderContainer` for your coupon rule.
- [ ] Commit your work and push it to your GitHub repo.

Check yourself

<details>
<summary>What problem does Riverpod solve that setState doesn't?</summary>

It shares state between widgets and screens without passing data and callbacks through constructors, and it rebuilds only the widgets that watch the changed value.
</details>

<details>
<summary>When do you use ref.watch and when ref.read?</summary>

`ref.watch` inside `build()` for values you display. `ref.read` inside callbacks like `onPressed` to change state.
</details>

▼ **Provider, StateProvider or NotifierProvider: when do you use each?**

Provider for read-only or calculated values. StateProvider for one simple value that changes. NotifierProvider for state with several operations and rules.

▼ **Why does `state.add(item)` not update the UI?**

It changes the existing list, so `state` still points to the same object and Riverpod sees no change. Assign a new list instead.

▼ **What does ProviderScope do?**

It stores the current value of every provider. Without it, no provider can be read.

NEXT SESSION

Session 6 · Working with APIs. Fetching real data from the internet with the `http` package, turning JSON into Dart classes, and showing loading, empty and error states with `FutureProvider`. You'll build a Joke app.